Lesson 6: An introduction to pointers

Pointers can be confusing, and at times, you may wonder why you would ever want to use them. The truth is, they can make some things much easier. For example, using pointers is one way to have a function modify a variable passed to it. It is also possible to use pointers to dynamically allocate memory allowing certain programming techniques, such as linked lists and resizable arrays. Pointers are what they sound like...pointers. They point to locations in memory. Picture a big jar that holds the location of another jar. In the other jar holds a piece of paper with the number 12 written on it. The jar with the 12 is an integer, and the jar with the memory address of the 12 is a pointer.

Pointer syntax can also be confusing, because pointers can both give the memory location and give the actual value stored in that same location. When a pointer is declared, the syntax is this: variable_type *name; Notice the *. This is the key to declaring a pointer, if you use it before the variable name, it will declare the variable to be a pointer.

As I have said, there are two ways to use the pointer to access information about the memory address it points to. It is possible to have it give the actual address to another variable, or to pass it into a function. To do so, simply use the name of the pointer without the *. However, to access the actual memory location, use the *. The technical name for this doing this is dereferencing.

In order to have a pointer actually point to another variable it is necessary to have the memory address of that variable also. To get the memory address of the variable, put the & sign in front of the variable name. This makes it give its address. This is called the address operator, because it returns the memory address.

For example:

```
#include <iostream>

using namespace std;

int main()
{
  int x;            // A normal integer
  int *p;           // A pointer to an integer

  p = &x;           // Read it, "assign the address of x to p"
  cin>> x;          // Put a value in x, we could also use *p here
  cin.ignore();
  cout<< *p <<"\n"; // Note the use of the * to get the value
  cin.get();
}
```

The cout outputs the value in x. Why is that? Well, look at the code. The integer is called x. A pointer to an integer is then defined as p. Then it stores the memory location of x in pointer by using the address operator (&). If you wish, you can think of it as if the jar that had the integer had a ampersand in it then it would output its name (in pointers, the memory address) Then the user inputs the value for x. Then the cout uses the * to put the value stored in the memory location of pointer. If the jar with the name of the other jar in it had a * in front of it would give the value stored in the jar with the same name as the one in the jar with the name. It is not too hard, the * gives the value in the location. The unasterisked gives the memory location.

Notice that in the above example, pointer is initialized to point to a specific memory address before it is used. If this was not the case, it could be pointing to anything. This can lead to extremely unpleasant consequences to the computer. You should always initialize pointers before you use them.

It is also possible to initialize pointers using free memory. This allows dynamic allocation of array memory. It is most useful for setting up structures called linked lists. This difficult topic is too complex for this text. An understanding of the keywords new and delete will, however, be tremendously helpful in the future.

The keyword new is used to initialize pointers with memory from free store (a section of memory available to all programs). The syntax looks like the example:

```
int *ptr = new int;
```

It initializes ptr to point to a memory address of size int (because variables have different sizes, number of bytes, this is necessary). The memory that is pointed to becomes unavailable to other programs. This means that the careful coder should free this memory at the end of its usage.

The delete operator frees up the memory allocated through new. To do so, the syntax is as in the example.

```
delete ptr;
```

After deleting a pointer, it is a good idea to reset it to point to 0. When 0 is assigned to a pointer, the pointer becomes a null pointer, in other words, it points to nothing. By doing this, when you do something foolish with the pointer (it happens a lot, even with experienced programmers), you find out immediately instead of later, when you have done considerable damage.